Aleatoriedade vs Pseudoaleatoriedade:

Demonstrando a importância do gerador de chaves em Criptoprafia Simétrica através de uma simulação simplicada em *Python*

Gisela Ceresér Kassick & Dr. Daniel Roberto Cassar

IV Encontro do Programa de Pós-Graduação em Matemática IGCE-UNESP-Rio Claro

gisela25026@ilum.cnpem.br

Introdução

A aleatoriedade é elemento central da criptografia moderna, sendo indispensável para a geração de chaves, nonces e vetores de inicialização. A segurança de sistemas criptográficos depende da produção de sequências imprevisíveis; portanto, a qualidade da fonte aleatória é determinante para a confidencialidade e integridade das comunicações (GENNARO, 2006).

Formalmente, uma sequência aleatória deve apresentar: (i) distribuição uniforme, (ii) independência entre elementos e (iii) imprevisibilidade a partir de subsequências observadas (ZHANG et al., 2021). A qualidade dessas sequências pode ser avaliada por métricas como entropia e testes de correlação, que identificam vieses ou dependências estatísticas.

Geradores verdadeiramente aleatórios (TRNGs) utilizam fenômenos físicos imprevisíveis, como ruído térmico, mas podem exigir técnicas de extração (ex.: funções hash) para reduzir vieses (CUCU LAURENCIU; COTOFANA, 2018; GENNARO, 2006). Já geradores pseudoaleatórios (PRNGs) expandem deterministicamente uma semente inicial. PRNGs convencionais não são adequados para criptografia, pois podem ter seu estado interno reconstruído (VIGNA, 2019), ao contrário dos CSPRNGs, projetados para manter imprevisibilidade mesmo sob análise parcial (D'APRANO, 2015).

Este trabalho investiga se diferenças na origem da aleatoriedade — física ou algorítmica — produzem variações estatísticas relevantes que impactam, de forma prática, a segurança de um sistema simples de criptografia simétrica. Espera-se que apenas fontes criptograficamente seguras mantenham desempenho estatístico e resistência prática adequados.

Metodologia

O estudo adotou abordagem experimental comparativa, com 10.000 execuções por fonte e sequências de 256 bits, integrando Arduino e Python via comunicação serial.

- **Fontes avaliadas:**
  - 'random' (PRNG não criptográfico)
  - 'secrets' (CSPRNG do sistema operacional)
  - Fonte física bruta (ruído elétrico via Arduino, mostrado na Figura 1)
  - Fonte física com extração por SHA-256
- **Análises realizadas:**
  - Entropia de Shannon
  - Autocorrelação
  - Teste qui-quadrado (uniformidade)

Por fim, os bits gerados derivaram chave AES-256 e IV de 128 bits para cifragem com Advanced Encryption Standard (AES) em modo CTR. A segurança prática foi avaliada por uma série de ataques: primeiramente, ataque por descoberta de seed, em seguida, força bruta limitada por viés estatístico e, por fim, ataque de força bruta limitado.

Os resultados estatísticos e o desempenho sob ataque foram comparados para avaliar o impacto da origem da aleatoriedade na segurança criptográfica.

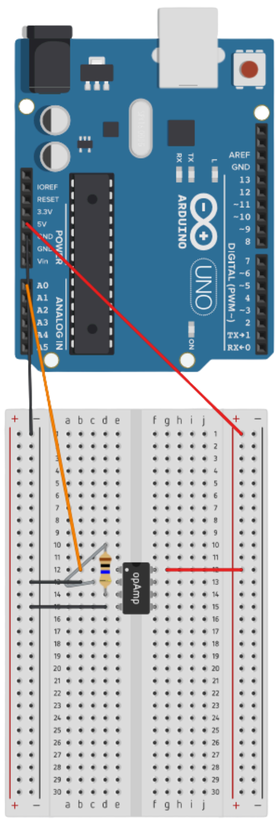

Figura 1.
FONTE: do autor, feito com *Tinkercad*

Resultados e Discussão

A partir de 10.000 iterações, foram comparadas métricas estatísticas das sequências geradas (Figura 2).

O gerador físico bruto apresentou entropia menor (≈ 0,972 bits/bit), viés na frequência de bits 1 (≈ 0,466) e autocorrelação positiva (≈ 0,191), indicando dependência entre bits consecutivos. Esse comportamento é esperado em fontes físicas devido a imperfeições do circuito e influências ambientais, como temperatura e variações de alimentação (CUCU; COTOFANA, 2018).

Após aplicar SHA-256 como extrator de entropia, a sequência apresentou frequência ≈ 0,5, entropia ≈ 0,997 bits/bit e autocorrelação ≈ −0,005, além de p-valores próximos de 0,5, indicando distribuição aproximadamente uniforme (GENNARO, 2006).

O random() apresentou boas métricas estatísticas (frequência ≈ 0,5, entropia ≈ 0,997, autocorrelação ≈ −0,004), mas foi comprometido nos ataques, pois utiliza o Mersenne Twister, cujo estado interno pode ser reconstruído a partir das saídas observadas (VIGNA, 2025).

Já secrets() apresentou métricas semelhantes e não foi comprometido, pois utiliza fontes de entropia do sistema operacional e mecanismos criptograficamente seguros (PYTHON SOFTWARE FOUNDATION, 2024).

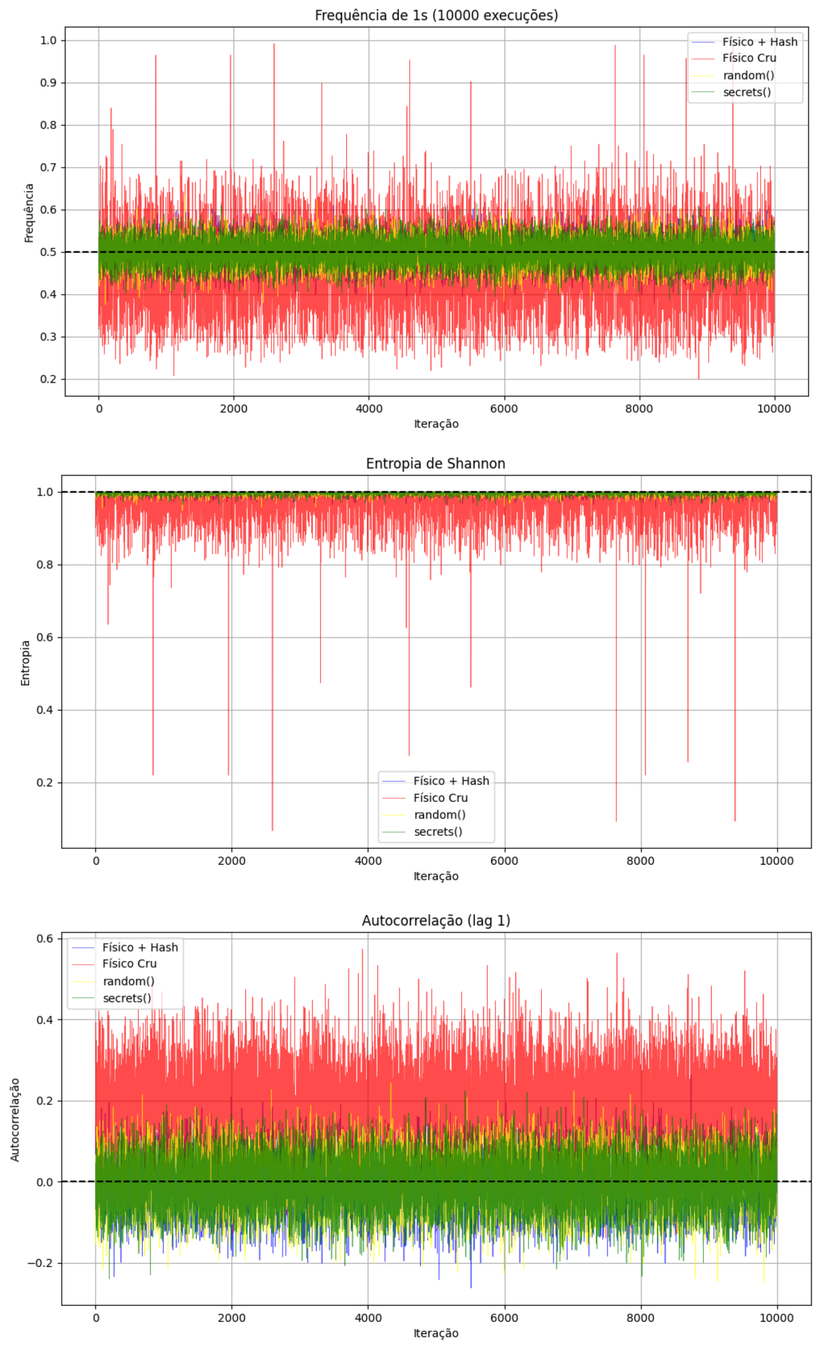

Figura 2.
FONTE: do autor

Considerações Finais

Em síntese, os resultados obtidos reforçam um princípio fundamental da criptografia: boas propriedades estatísticas não são suficientes para garantir segurança criptográfica. O gerador random() apresentou métricas estatísticas praticamente ideais, mas permaneceu vulnerável a ataques devido à sua natureza determinística. Por outro lado, a fonte física bruta apresentou imperfeições experimentais, como viés e autocorrelação, que podem reduzir a entropia efetiva da sequência gerada. A aplicação de funções hash criptográficas demonstrou ser um mecanismo eficaz para mitigar essas imperfeições e produzir sequências mais adequadas para aplicações de segurança. Dessa forma, os resultados evidenciam a importância de utilizar algoritmos e arquiteturas especificamente projetados para criptografia ao construir sistemas de geração de números aleatórios destinados a aplicações seguras.

Referências

BLAZHEVSKI, D.; et al. Modes of Operation of the AES Algorithm. 2012.

CUCU LAURENCIU, N.; COTOFANA, S. D. Low Cost and Energy, Thermal Noise Driven, Probability Modulated Random Number Generator. Delft University of Technology, 2018.

D'APRANO, S. PEP 506 – Adding a Secrets Module to the Standard Library. 2015. Disponível em: https://peps.python.org/pep-0506/. Acesso em: 26 fev. 2026.

GENNARO, R. Randomness in Cryptography. IEEE Security & Privacy, v. 4, n. 2, p. 64-67, 2006.

KATZ, J.; LINDELL, Y. Introduction to Modern Cryptography. 2007.

MENGDI, Z. et al. Overview of Randomness Test on Cryptographic Algorithms. Journal of Physics: Conference Series, v. 1861, n. 1, 012009, 2021.

PYTHON SOFTWARE FOUNDATION. random — geração de números pseudoaleatórios. Disponível em: https://docs.python.org/3/library/random.html. Acesso em: 26 fev. 2026.

PYTHON SOFTWARE FOUNDATION. secrets — geração de números seguros para gerenciamento de dados sensíveis. Disponível em: https://docs.python.org/3/library/secrets.html. Acesso em: 26 fev. 2026.

PYTHON SOFTWARE FOUNDATION. hashlib — algoritmos de hash seguros. Disponível em: https://docs.python.org/3/library/hashlib.html. Acesso em: 26 fev. 2026.

PYCA CRYPTOGRAPHY. Cryptography Documentation. Disponível em: https://cryptography.io. Acesso em: 26 fev. 2026.

VIGNA, S. It Is High Time We Let Go of the Mersenne Twister. arXiv:1910.06437, 2019. Disponível em: https://arxiv.org/abs/1910.06437. Acesso em: 26 fev. 2026

WALTER, Daniela; BÜLAU, André; ZIMMERMANN, André. Review on excess noise measurements of resistors. Sensors, v. 23, n. 3, p. 1107, 2023. DOI: https://doi.org/10.3390/s23031107

Link do GitHUb do projeto: https://github.com/GiselaCK/pseudo-vs-physical-randomness